01 / 最常考的概念與 Tensor

先看 shape，再看 dtype、device；不要先猜 batch size 有問題。

Tensor 操作：功能不同，別混在一起

shape / size()：各軸大小；ndim：軸的數量；size(0)：第 0 軸大小。
[N, C, H, W] = 批次、通道、高、寬；[N, K] = 批次、類別分數。

```
x = torch.randn(128, 3, 32, 32)
x.shape           # torch.Size([128, 3, 32, 32])
x.ndim            # 4
x.size(0)         # 128
x.unsqueeze(0)    # add an axis at position 0
x.permute(0, 2, 3, 1)  # NCHW -> NHWC
x.flatten(1)      # [128, 3072], preserve batch axis
x.detach().cpu().numpy()  # tensor -> NumPy for plotting
```

argmax / max / item / clamp

```
logits = torch.tensor([[1., 4., 2.], [5., 0., 1.]])
logits.argmax(dim=1)    # tensor([1, 0]): class per sample
logits.argmax(dim=0)    # tensor([1, 0, 0]): sample per class
values, indices = logits.max(dim=1)
correct = (logits.argmax(1) == labels).sum().item()
x = torch.tensor([-0.2, 0.3, 1.5]).clamp(0, 1)
# tensor([0.0, 0.3, 1.0])
```

argmax 回傳索引；max(dim) 回傳最大值與索引；dim 一次指定一個軸。
item() 只適用單一元素 Tensor，取出 Python 數字；clamp 是截斷，不是縮放。

train / eval / no_grad：三者不能互相取代

model.train()：Dropout 啟用；BatchNorm 使用批次統計並更新累積統計。
model.eval()：Dropout 關閉；BatchNorm 預設使用累積統計。
torch.no_grad()：不建立梯度追蹤；驗證／推論通常與 eval() 一起用。
eval() 不會自動關閉梯度；train() 本身不會更新權重。
zero_grad() 清梯度；backward() 算梯度；optimizer.step() 更新權重。
inplace=True：直接改原 Tensor；可能影響 backward 所需的中間值。

分類輸出與 loss 配對

多類別單標籤：logits [N,K] + CrossEntropyLoss；labels [N]、long。
訓練前不要再加 softmax；要機率才 softmax(dim=1)，取類別直接 argmax。
二元／多標籤：BCEWithLogitsLoss 接 logits；target 為同形狀浮點數。
二元／多標籤要機率時才 sigmoid；多標籤各類別獨立，可同時為真。

02 / 資料與 CNN 尺寸

FashionMNIST：[1,28,28]；CIFAR10：[3,32,32]。

資料切分與 DataLoader

```
import torch
from torch import nn
from torch.utils.data import DataLoader, random_split
from torchvision import datasets, transforms
from pathlib import Path

torch.manual_seed(42)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
tfm = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,)*3, (0.5,)*3),
])
full = datasets.CIFAR10("data", train=True, download=True, transform=tfm)
train_set, val_set = random_split(
    full, [40000, 10000], generator=torch.Generator().manual_seed(42))
train_loader = DataLoader(train_set, batch_size=128, shuffle=True)
val_loader = DataLoader(val_set, batch_size=128, shuffle=False)
```

ToTensor 將這些資料集的 uint8 圖片轉成 [C,H,W] 浮點數，範圍 0~1。
Normalize：(x - mean) / std；訓練、驗證與推論的正規化必須一致。
FashionMNIST 改資料集名稱、Normalize((0.5,), (0.5,))、切分 [50000,10000]。
驗證集用來選模型；測試集留作最終評估。隨機增強通常只用在訓練集。

可切換兩種資料集的模型

```
class CNN(nn.Module):
    def __init__(self, in_channels=3, image_size=32, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(in_channels, 32, 3, padding=1),
            nn.BatchNorm2d(32), nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1),
            nn.BatchNorm2d(64), nn.ReLU(), nn.MaxPool2d(2),
        )
        s = image_size // 4
        self.classifier = nn.Sequential(
            nn.Flatten(), nn.Linear(64*s*s, 256),
            nn.ReLU(), nn.Dropout(0.5), nn.Linear(256, num_classes))
    def forward(self, x):
        return self.classifier(self.features(x))

model = CNN(3, 32).to(device)    # CIFAR10
# model = CNN(1, 28).to(device) # FashionMNIST
```

尺寸／參數速算

Conv 輸出高 = floor((H + 2P - D(K-1) - 1) / S + 1)，寬同理。
3x3、P=1、S=1、D=1：高寬不變。MaxPool2d(2)：本例高寬減半。
CIFAR：32 → 16 → 8；FashionMNIST：28 → 14 → 7。
Conv 參數 = Cout × (Cin/groups × Kh × Kw + 1)，+1 為 bias。
Linear 參數 = out × (in + 1)；BatchNorm 可訓練參數通常為 2C。
兩層 3x3、stride=1 的感受野為 5x5；加深不保證準確率增加。

03 / 訓練＋驗證完整核心

沿用第 2 頁的 model、device、train_loader、val_loader；放在 main() 中執行。

```
out_dir = Path("result")
out_dir.mkdir(parents=True, exist_ok=True)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode="min", factor=0.5, patience=2)
best = float("inf")
epochs = 5
csv_path = out_dir / "loss.csv"
csv_path.write_text("epoch,train_loss,val_loss,train_acc,val_acc\n")

for epoch in range(1, epochs + 1):
    model.train()
    tr_sum, tr_ok, n = 0.0, 0, 0
    for images, labels in train_loader:
        images = images.to(device)
        labels = labels.to(device).long()
        optimizer.zero_grad()
        logits = model(images)
        loss = criterion(logits, labels)
        loss.backward()
        optimizer.step()
        bs = labels.size(0)
        tr_sum += loss.item() * bs
        tr_ok += (logits.argmax(1) == labels).sum().item()
        n += bs
    train_loss, train_acc = tr_sum / n, tr_ok / n

    model.eval()
    va_sum, va_ok, m = 0.0, 0, 0
    with torch.no_grad():
        for images, labels in val_loader:
            images = images.to(device)
            labels = labels.to(device).long()
            logits = model(images)
            loss = criterion(logits, labels)
            bs = labels.size(0)
            va_sum += loss.item() * bs
            va_ok += (logits.argmax(1) == labels).sum().item()
            m += bs
    val_loss, val_acc = va_sum / m, va_ok / m
    scheduler.step(val_loss)
    with csv_path.open("a") as f:
        f.write(f"{epoch},{train_loss},{val_loss},"
                f"{train_acc},{val_acc}\n")
    print(epoch, train_loss, val_loss, train_acc, val_acc)
    if val_loss < best:
        best = val_loss
        torch.save({
            "epoch": epoch, "model_state": model.state_dict(),
            "optimizer_state": optimizer.state_dict(),
            "scheduler_state": scheduler.state_dict(),
            "val_loss": val_loss,
        }, out_dir / "best.pth")
```

三個必記原因

loss.item() × bs：此處 CE 預設平均、無 class weight／ignored label；
先還原批次總 loss，再除以總樣本數，避免最後小批次造成平均錯誤。
梯度預設會累積，所以每個 batch 更新前要 zero_grad()。
下個 epoch 開頭要切回 train()；驗證不做 backward()、step()。

04 / 載入、推論與畫圖

CNN_template.py 放架構；train.py 訓練與驗證；predict.py 載入模型並推論。

Checkpoint：先建立相同架構，再載入權重

```
from CNN_template import CNN
import matplotlib.pyplot as plt

model = CNN(3, 32).to(device)  # same architecture as page 2
ckpt = torch.load("result/best.pth", map_location=device,
                  weights_only=True)
model.load_state_dict(ckpt["model_state"])
model.eval()
test_set = datasets.CIFAR10("data", train=False,
                            download=True, transform=tfm)
img_t, label = test_set[0]
with torch.no_grad():
    logits = model(img_t.unsqueeze(0).to(device))  # [1,3,32,32]
    probs = logits.softmax(dim=1)
    conf, pred = probs.max(dim=1)
print("label:", label, "pred:", pred.item(), "prob:", conf.item())
```

state_dict 包含模型參數與持久 buffer（例如 BN 統計），不包含完整架構。
若存的是 torch.save(model.state_dict(), path)，載入後直接傳給 load_state_dict。
若存的是上頁字典，則取 ckpt["model_state"]；兩種格式不要混淆。
推論不需 optimizer；要接續訓練才恢復 optimizer、scheduler、epoch 與 best。

還原圖片：數值範圍和維度是兩個不同問題

```
def denorm_img(x):
    x = x.detach().cpu()
    x = (x * 0.5 + 0.5).clamp(0, 1)  # matches this sheet's tfm
    if x.shape[0] == 1:
        x = x[0]               # [1,H,W] -> [H,W]
    else:
        x = x.permute(1, 2, 0)  # [3,H,W] -> [H,W,3]
    return x.numpy()

fig, ax = plt.subplots()
img = denorm_img(img_t)
ax.imshow(img, cmap="gray" if img.ndim == 2 else None)
ax.set_title(f"GT: {label} / Pred: {pred.item()}")
ax.axis("off")
fig.savefig("result/prediction.png", dpi=150, bbox_inches="tight")
plt.close(fig)
```

MNIST 能直接畫是因為取出單通道後變成 [H,W]；RGB 需要 [H,W,3]。
clamp 限制範圍；numpy 轉資料型別。RGB 不可用 x[0]，那會丟掉兩個通道。

CSV 畫 loss 曲線

```
import pandas as pd
history = pd.read_csv("result/loss.csv")
fig, ax = plt.subplots()
ax.plot(history["epoch"], history["train_loss"], label="train")
ax.plot(history["epoch"], history["val_loss"], label="validation")
ax.set(xlabel="epoch", ylabel="loss", title="Loss Curve")
ax.legend()
fig.tight_layout()
fig.savefig("result/loss_curve.png")
plt.close(fig)
```

05 / 報錯速查與考前檢查

先讓一個 batch 跑通，再開始長時間訓練。

常見錯誤 → 原因 → 修正

expected ... 1 channels, but got 3
→ CIFAR 是 RGB；第一層改 Conv2d(3, ...)，不是調 batch size。
mat1 and mat2 shapes cannot be multiplied
→ Flatten 數量與 Linear 不一致；印 features(x).shape，重算 C×H×W。
size mismatch ... load_state_dict
→ checkpoint 與當前架構不同；用原架構或新模型重訓。
strict=False 不是跳過同名參數尺寸不符的解法。
Invalid shape (3,32,32) for image data
→ imshow 要 [32,32,3]；使用 permute(1,2,0)。
Expected all tensors to be on the same device
→ model、images、labels 都要移到同一個 device。
Expected ... Long / target out of bounds
→ CE 類別標籤使用 long，範圍 0 到 num_classes-1，形狀 [N]。
CUDA: no kernel image is available
→ PyTorch 編譯版本不支援 GPU 架構；is_available=True 不代表能運算。
你的 RTX 5060 使用已確認的 torch 2.7.1 + cu128 組合。
CUDA out of memory
→ 減小 batch_size；驗證用 no_grad；統計 loss 用 item() 避免保留計算圖。
No space left on device（安裝時）
→ 檢查 df -h；/tmp 可能比磁碟小，改 TMPDIR 到家目錄。

WSL 環境速查

```
source .venv/bin/activate
which python
python --version
python -m pip install -r requirements.txt
python -m pip check
nvidia-smi
# If installation exhausts /tmp:
mkdir -p "$HOME/pip-tmp"
TMPDIR="$HOME/pip-tmp" python -m pip install -r requirements.txt
```

開始作答前的 6 個確認

1. 資料集正確？FashionMNIST [1,28,28]／CIFAR10 [3,32,32]。
2. train／val／test 切分正確？標籤名稱與資料集的 classes 對應。
3. 模型輸出 [N,10]？CrossEntropyLoss 前沒有 softmax。
4. 訓練五步齊全？zero_grad → forward → loss → backward → step。
5. 驗證 eval + no_grad？以樣本數加權統計？儲存最佳 checkpoint。
6. 推論架構、正規化、checkpoint 路徑一致？輸出目錄已建立。

官方查閱入口（需要時）

docs.pytorch.org/docs/stable/ | docs.pytorch.org/vision/stable/
pytorch.org/get-started/previous-versions/
此小抄以單標籤影像分類為主；第 2～4 頁使用同一個示範架構。